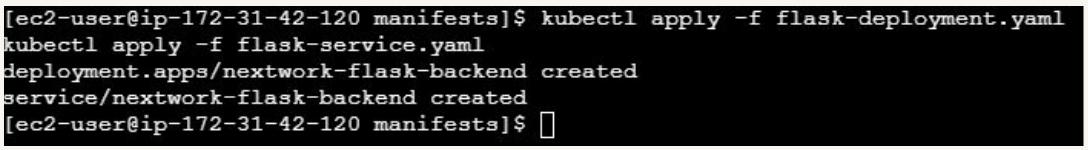
Deploy Backend with Kubernetes

Mohammed Dawoud Mota



Introducing Today's Project!

In this project I am setting up and deploying the backend of an application onto a Kubernetes cluster using Amazon EKS. I will prepare the environment, install the necessary tools, build and store my container image, create the Kubernetes manifests, and finally deploy the backend so it can run reliably inside the cluster.

Tools and concepts

The key tools and concepts in this project centered around deploying a backend application using Kubernetes and AWS. I worked with Amazon EKS to manage the Kubernetes cluster, Amazon EC2 to run the commands and build the environment, and Amazon ECR to store my container image. Docker was essential for building the image from the backend code, and kubectl allowed me to interact with the cluster and apply my Deployment and Service manifests. I also used eksctl to create and configure the EKS cluster. Altogether these tools and concepts formed the workflow that let me package the backend, store it securely, define how it should run, and deploy it reliably into Kubernetes.

Project reflection

It took me about 30 minutes to complete this project from start to finish. The workflow was structured in a way that made each step build naturally on the previous one, so even though there were several moving parts like setting up the EKS cluster, building and pushing the container image, and creating the Kubernetes manifests, the overall process stayed smooth and manageable within that timeframe.

Project Set Up

Kubernetes cluster

To set up my Kubernetes cluster, I launched a new EC2 instance in my AWS account and connected to it using EC2 Instance Connect so I could run commands directly from the terminal. I installed eksctl, which is the command line tool that simplifies creating and managing EKS clusters, and verified the installation. Since the EC2 instance initially has no permissions, I created an IAM role with AdministratorAccess and attached it to the instance so it could interact with AWS services. With the environment prepared and the correct permissions in place, I used eksctl from the EC2 terminal to create the EKS cluster that will host my backend application.

Backend code

I retrieved the backend code by installing Git on my EC2 instance, configuring it with my name and email, and then cloning the team member's GitHub repository directly into the instance. After opening the repository page in a browser, I copied the HTTPS clone URL and used the git clone command in the terminal to pull down the full nextwork-flask-backend project. Once the clone completed, I verified the files by navigating into the new directory and listing its contents, which included the app.py file, the Dockerfile, the requirements.txt file, and the README. This gave me a complete and ready-to-use copy of the backend code that I would later containerize and deploy.

Container image

I built a container image because Kubernetes needs a packaged and consistent version of my backend application that it can pull and run inside the cluster. The image includes all the code, dependencies and instructions defined in the Dockerfile, which ensures every container created by Kubernetes behaves the same across different environments. Without a container image, Kubernetes would not have a reliable blueprint to create identical pods, making the

Manifest files

A Kubernetes manifest is a file that provides Kubernetes with the instructions it needs to run and manage my application inside the cluster. It defines the desired state of my backend, including which container image to use, how many replicas to create, how the pods should be labeled, and how the application should be exposed. Manifests are helpful because they make deployment consistent and repeatable, allowing Kubernetes to automatically set up and maintain my application without requiring manual configuration each time.

A Deployment manifest defines how Kubernetes should run and manage the backend application inside the cluster. It specifies the number of replicas that should be created, the labels that identify the pods, and the exact container image that Kubernetes needs to pull from ECR. By describing the desired state of the application, the Deployment ensures that Kubernetes keeps the backend running consistently, replaces pods if they fail, and maintains the correct number of instances across the cluster.

A Kubernetes Service acts as the traffic manager for my application inside the cluster. It provides a stable way for other services or external users to reach the pods running my backend, even though those pods can change or restart at any time. The Service watches for pods with the correct labels and routes incoming traffic to them on the right port, which ensures my backend stays reachable and consistent no matter what is happening behind the scenes in the cluster.

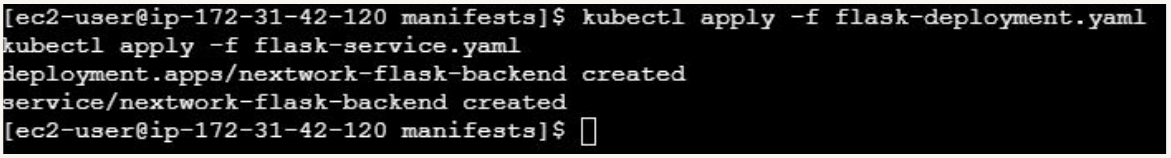

GNU nano 8.3
flask-deployment.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
name: nextwork-flask-backend
namespace: default
spec:
replicas: 3
selector:
matchLabels:
app: nextwork-flask-backend
template:
metadata:
labels:
app: nextwork-flask-backend
spec:
containers:
- name: nextwork-flask-backend
image: 289654514139.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-backend:latest
ports:
- containerPort: 8080
[Read 21 lines]
^G Help
^X Exit
^O Write Out
^R Read File
^W Where Is
^\ Replace
^K Cut
^U Paste
^T Execute
^J Justify
^C Location
^/ Go To Line
M-U Undo
M-E Redo
M-A Set Mark
M-6 Copy
M-] To Bracket
^B Where Was

Backend Deployment!

To deploy my backend application, I installed kubectl on my EC2 instance and then used it to apply the Kubernetes manifests I created earlier. Once kubectl was set up, I ran the commands to apply both my Deployment and Service YAML files. Applying the Deployment told Kubernetes to pull my container image from ECR and spin up the pods, and applying the Service exposed those pods so the backend could be reached. In short, I used kubectl to send my manifests to the cluster, and Kubernetes handled the rest.

kubectl

Kubectl is the command-line tool I use to interact with my Kubernetes cluster. It lets me apply my Deployment and Service manifests, check the status of my pods, and manage anything running inside EKS. Without kubectl, my cluster would be up and running, but I wouldn't have a way to actually deploy my backend or control how Kubernetes behaves.



The place to learn & showcase your skills

Check out nextwork.ai for more projects